

Generative Models for Cat Image Synthesis

Paweł Pozorski¹, Paweł Florek¹

¹ Warsaw University of Technology, Poland

{pawel.pozorski, pawel.florek}.stud@pw.edu.pl

Abstract

We present a systematic comparison of three generative paradigms—generative adversarial networks (GANs), variational autoencoders (VAEs), and denoising diffusion probabilistic models—for unconditional synthesis of cat images. The study emphasizes experimental reproducibility through hierarchical YAML configuration, deterministic pseudorandom seeding, and multi-run variance characterization. Methodological aspects cover dataset acquisition, fixed-resolution preprocessing, family-specific objectives, optimization budgets, early stopping, a two-phase model-specific hyperparameter sweep with best-baseline carry-over, and evaluation via Fréchet Inception Distance (FID), LPIPS/MS-SSIM diversity, and ten-image latent interpolation. A single software stack supports training with checkpoint resumption, MLflow logging, and sweep summaries on disk. An exploratory class-conditional extension on cats-versus-dogs imagery is outlined separately. Quantitative results are deferred pending completion of the full matrix and sweep.

Index Terms: generative models, GAN, VAE, diffusion, image synthesis, mode collapse, latent interpolation

1. Introduction

Generative modeling seeks to approximate high-dimensional data distributions such that novel samples are statistically faithful to the training domain while exhibiting plausible structure. In image synthesis, three families have become dominant: adversarial training [1], amortized variational inference with continuous or discrete latents [2, 3], and iterative denoising diffusion [4]. We investigate all three in a unified setting—synthesis of cat imagery at moderate resolution—so that differences in sample quality, diversity, optimization stability, and sensitivity to initialization may be attributed primarily to the modeling paradigm rather than to unrelated preprocessing or infrastructure choices.

The experimental setting is deliberately bounded by a fixed local setup, which keeps the training and evaluation protocol consistent across runs. Within these limits, we nevertheless adhere to standard quantitative reporting (FID, complementary perceptual metrics) and to qualitative diagnostics of known failure modes, notably mode collapse in adversarial training and oversmoothing in likelihood-based models.

For each family we instantiate a baseline and a full enhanced configuration informed by the literature (e.g., self-attention and spectral normalization in GANs [5, 6], vector quantization in VAEs [3], refined schedules and sampling in diffusion [7]). A separate hyperparameter sweep systematically varies model-specific parameters on both variants to quantify the influence of individual design choices. All primary com-

parisons are repeated across multiple independent seeds so that point estimates of performance are accompanied by dispersion, reflecting the stochastic nature of non-convex optimization in deep generative modeling.

1.1. Objectives

The study is organized around the following objectives:

1. to synthesize cat images under each of the three paradigms with matched preprocessing and comparable training budgets;
2. to contrast baseline and full enhanced variants *within* each paradigm, and to investigate the influence of model-specific hyperparameters via a dedicated sweep;
3. to quantify distributional fidelity via FID [8] and to characterize diversity using perceptual and structural similarity measures [9, 10];
4. to quantify sensitivity of final metrics to random initialization and training stochasticity across seeds;
5. to construct latent interpolation trajectories for qualitative assessment of the learned representation;
6. to conduct a secondary, exploratory analysis of class-conditional generation on a cats-versus-dogs dataset.

1.2. Contributions

The principal contributions of this work are as follows:

- a modular, strongly typed training and evaluation codebase that treats GAN, VAE, and diffusion models behind a common trainer interface, reducing implementation confounds when comparing variants;
- model configurations chosen to remain practical within the shared training setup without abandoning established architectural ideas from the literature;
- automated orchestration of a multi-seed, multi-variant experiment matrix with consistent logging and checkpoint semantics;
- integration of early stopping and resumable training, which is particularly relevant for interrupted runs;
- an optional class-conditional GAN extension for cross-species exploratory synthesis, clearly delineated from the primary cat-only benchmark.

2. Methodology

2.1. Datasets and acquisition

Two corpora are used:

- **Cats (primary):** the Kaggle `crawford/cat-dataset`

collection for unconditional cat synthesis. Images are expected under a project-local root (default `data/cat-dataset`). If the directory is empty and automated download is enabled, credentials in `.env` (`KAGGLE_USERNAME`, `KAGGLE_KEY`) are used to fetch data; downloaded files are copied into the project root so paths remain stable for exploratory data analysis.

- **Cats versus dogs (extension):** the Kaggle `dogs-vs-cats` competition imagery, used only for an exploratory class-conditional GAN. This corpus is not auto-downloaded; authors place extracted files under a configured root. Labels are inferred from filenames (`cat.*`, `dog.*`) or parent folder names.

Handling heterogeneous resolutions. Source JPEGs vary in size. All models use a *single* training resolution: images are resized to 64×64 pixels (bicubic resampling via the data pipeline), converted to RGB, and linearly scaled to $[-1, 1]$ channel-wise. This unifies the domain across GAN, VAE, and diffusion heads and avoids architecture-specific multi-scale loaders.

2.2. Preprocessing, augmentation, and loading

Training uses PyTorch `DataLoaders` with configuration-driven `batch_size`, `shuffle`, `drop_last`, and `num_workers`. Dataloader workers receive deterministic seeds derived from the run seed where supported, to reduce run-to-run drift under multiprocessing. No heavy color augmentation is applied in the *baseline* dataloaders; the full GAN variant may incorporate discriminator-side augmentations as specified in model configuration.

2.3. Model families, objectives, and implemented variants

The primary matrix compares a *baseline* and a *full* variant per family (Sec. 2.7). Unless noted, both use the same optimizer family and learning-rate schedule (Sec. 2.4).

2.3.1. GAN family

Baseline (DCGAN). Convolutional generator with transposed strided convolutions and batch normalization; convolutional discriminator without spectral normalization or self-attention [11]. Training uses a binary cross-entropy / sigmoid objective with alternating generator and discriminator updates.

Full (SA-DCGAN+). Adds self-attention at the 16×16 feature stage in both networks [5], spectral normalization on the discriminator [6], hinge loss (generator minimizes $-\mathbb{E}[D(G(z))]$, discriminator uses the hinge margin form), and lazy R1 regularization on real inputs with configurable `rl_gamma` and `rl_interval`. Latent vectors are Gaussian, spatially broadcast as in the DCGAN-style layout.

2.3.2. VAE family

Baseline. Gaussian VAE with reparameterized latents [2]. The objective combines mean L1 reconstruction loss and a KL term weighted by `kl_weight`.

Full (VQ-VAE). Vector-quantized bottleneck [3]; the codebook is updated with exponential moving averages (EMA). The training loss sums L1 reconstruction and a commitment term tied to the codebook module. After the encoder-decoder stage, a lightweight convolutional *PixelCNN-style* prior is trained on discrete code indices to support sampling; the prior uses cross-entropy next-index prediction on the latent grid.

2.3.3. Diffusion family

Baseline (DDPM). Discrete-time DDPM [4] with a *linear* $\{\beta_t\}$ schedule and fixed variance in the reverse updates. A time-conditioned U-Net predicts additive noise ϵ . At each training step, a random t is drawn and the network minimizes the mean squared error between ϵ and $\epsilon_\theta(x_t, t)$.

Full (improved configuration). Same U-Net backbone and noise-prediction objective, but the forward process uses a *cosine* β schedule [7]. Configuration exposes `learn_sigma` for the improved-DDPM family; the current training loop uses the shared ϵ -MSE loss, while reverse sampling follows the standard ancestral DDPM update with the trained schedule.

2.4. Optimization and compute budgets

Unless swept, optimization uses Adam with learning rate 2×10^{-4} and $(\beta_1, \beta_2) = (0.5, 0.999)$. Epoch and batch budgets are fixed per family in the orchestration layer: GAN runs 120 epochs at batch size 64; VAE runs 100 epochs at batch size 64; diffusion runs 180 epochs at batch size 32 (reduced to respect memory limits for long reverse chains). Experiments target Apple Silicon with the PyTorch MPS backend; the code path remains valid on CUDA.

2.5. Training loop, logging, and checkpointing

All trainers inherit a shared abstraction that:

- sets Python, NumPy, and PyTorch seeds from `project.seed`;
- logs scalar metrics each epoch to MLflow (local `mlruns` store);
- serializes joint model and optimizer state to disk under `training.checkpoint_dir`;
- supports resume when `resume_if_exists` is true and a checkpoint path exists;
- writes `finalmetrics.json` in the checkpoint directory after training, storing the last epoch’s reported losses for automated sweep analysis.

Orchestration scripts (`make train-matrix`, `make sweep`) expand Cartesian products of configuration and seeds without manual bookkeeping.

2.6. Early stopping

Early stopping is enabled by default. The monitored metric is chosen per family: `gan/d_loss` for GANs, `vae/total_loss` for VAEs, and `diffusion/loss` for diffusion. Training stops when the monitored quantity fails to improve beyond `min_delta` for patience consecutive epochs (mode: `minimize`). This limits wasted compute on diverged or plateaued runs on resource-constrained hardware.

2.7. Primary experimental matrix

The cat-only *reference* matrix trains baseline and full variants for GAN, VAE, and diffusion, each with seeds $\{42, 123, 7\}$, yielding $2 \times 3 \times 3 = 18$ runs with fixed default hyperparameters. This layer establishes comparable baselines before family-specific sweeps.

2.8. Hyperparameter influence study

To address the course requirement on hyperparameter effects, we run a *two-phase*, model-specific sweep (learning rate and

batch size are held fixed unless they become the explicit object of study).

Phase 1—baseline grid: Each family baseline is trained on its baseline hyperparameter grid with three seeds per configuration. The configuration minimizing the mean (over seeds) of the monitored final training loss—read from `final_metrics.json`—is designated *best baseline*.

Phase 2—full grid with carry-over: Dot-list overrides from the best baseline (e.g., best `latent_dim` for DCGAN) are prepended to every full-variant configuration before training, so the full model is evaluated in the latent/capacity regime that performed best under the simpler baseline.

Grids (35 configurations):

- GAN baseline: `latent_dim` $\in \{64, 128, 256\}$ (3).
- GAN full: `rl_gamma` $\in \{1, 10, 50\}$, `rl_interval` $\in \{8, 16\}$ (6).
- VAE baseline: `latent_dim` $\times$ `kl_weight` $\in \{64, 128, 256\} \times \{0.5, 2.0\}$ (6).
- VAE full: `num_embeddings` $\in \{256, 512\}$, `embedding_dim` $\in \{64, 128\}$, `ema_decay` $\in \{0.95, 0.99\}$ (8).
- Diffusion baseline: `training_timesteps` $\in \{500, 1000\}$ and evaluation-oriented `sample_steps` $\in \{250, 500\}$ (documented for analysis and downstream sampling scripts) (4).
- Diffusion full: `ddim_steps` $\in \{50, 100\}$, `ddim_eta` $\in \{0.0, 0.2\}$, `self_cond_prob` $\in \{0.25, 0.5\}$ (8), in addition to carry-over baseline keys.

Each configuration is trained with three seeds: $35 \times 3 = 105$ sweep runs. Summaries are written under `results/hparam_sweep/`. **Note:** training losses are used as the sweep objective for automation; final model ranking for the report still uses held-out FID and diversity metrics (Sec. 2.9).

2.9. Evaluation protocol

Post-training evaluation follows a shared protocol:

- **FID:** compare a fixed number of generated samples (configurable, default 5000) against real images using a pretrained Inception feature extractor [8], batched for memory.
- **Diversity / mode collapse:** pairwise LPIPS [9] on generated batches and multiscale SSIM (MS-SSIM) [10] to detect collapse (low LPIPS spread, inflated MS-SSIM).
- **Latent interpolation:** linear interpolation between two latent codes (or noise endpoints for the generative model in use), yielding eight intermediate latents between two anchor samples, for ten images in total, as required by the assignment brief.
- **Seed uncertainty:** report mean and standard deviation (or median and IQR) of each metric across the three seeds wherever applicable.

Qualitative panels and metric tables are produced by dedicated scripts and exported for the manuscript.

2.10. Cats-versus-dogs extension

The extension trains a class-conditional GAN [1]: class embeddings are injected into the generator and discriminator (label maps / embeddings as implemented in the codebase). We treat cross-class generation as exploratory: the emphasis is on whether samples resemble distinct species or blended attributes,

not on competition-level accuracy.

3. Related Work and Benchmark Context

Adversarial generation. GANs [1] formulate learning as a two-player game between a generator and a discriminator. DCGAN [11] demonstrated that careful architectural constraints yield stable synthesis of natural images. Nonetheless, adversarial training remains sensitive to hyperparameters and susceptible to mode collapse. Spectral normalization [6] constrains the Lipschitz constant of the discriminator, while self-attention [5] captures long-range spatial dependencies; both appear in our full SA-DCGAN+ configuration, and the hyperparameter sweep quantifies the effect of regularization strength.

Latent-variable and quantized representations. VAEs [2] optimize a tractable lower bound on the marginal log-likelihood with a Gaussian latent prior, often producing slightly blurred reconstructions. VQ-VAE [3] replaces continuous latents with discrete codes drawn from a learned codebook, improving fidelity and enabling powerful autoregressive priors. Our study contrasts the Gaussian baseline against the full VQ-VAE with EMA codebook and sweeps codebook and KL-related parameters.

Diffusion modeling. DDPMs [4] define a forward noising process and learn its reverse as a sequence of denoising steps. Subsequent work refined variance schedules and parameterizations to improve sample quality and efficiency [7]. Diffusion typically demands more function evaluations per sample than single-step generators but offers comparatively stable optimization.

Evaluation methodology. FID [8] remains a de facto standard for comparing generative models on natural images. LPIPS [9] measures perceptual distance using deep features; MS-SSIM [10] quantifies multiscale structural agreement. Jointly, these metrics provide complementary views on fidelity and intra-batch diversity.

Positioning. Although Interspeech primarily addresses speech and spoken-language processing, the present manuscript applies the same experimental hygiene expected in rigorous machine-learning research—explicit seeds, versioned configurations, and quantitative metrics—to a vision benchmark chosen for visual clarity of failure modes and ease of qualitative inspection.

4. Experimental Design and Reporting

Compute environment. See Sec. 2.4 for hardware targets (Apple Silicon / MPS) and optimizer defaults.

Primary training matrix. The reference 18-run matrix is specified in Sec. 2.7; it precedes the hyperparameter sweep and uses default YAML configurations.

Hyperparameter sweep. The two-phase sweep (105 runs, best-baseline carry-over) is specified in Sec. 2.8; JSON summaries are emitted under `results/hparam_sweep/`.

Optimization and early stopping. Patience-based early stopping and per-family monitors are documented in Sec. 2.6.

Data management. Dataset roots, Kaggle acquisition, and resolution policy appear in Sec. 2.1.

Post-training analysis. FID, diversity metrics, interpolation, and seed aggregation follow Sec. 2.9.

Upcoming results section. Section 5 will consolidate tabular comparisons, seed-sensitivity visualizations, hyperparameter slice plots, and qualitative panels, linking observed artifacts

(e.g., collapse, texture loss, color inconsistency) to the logged quantitative scores and to specific configuration choices.

5. Results

Quantitative metrics, statistical summaries across seeds, hyperparameter influence plots, and qualitative figures will be inserted after the full training matrix and sweep are finalized.

6. Conclusion

A discussion interpreting the empirical findings in light of Sec. 3, together with limitations of the experimental setup and directions for future work, will be added once results are available.

7. AI Disclosure

This manuscript was drafted with assistance from generative language models for phrasing and organization. All scientific judgments, implementation decisions, and interpretations remain the authors' responsibility.

8. References

- [1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf
- [2] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [3] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, 2017.
- [4] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, 2020.
- [5] H. Zhang, I. Goodfellow, D. Metaxas, and A. Odena, "Self-attention generative adversarial networks," in *Proc. Int. Conf. Machine Learning*, 2019.
- [6] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. Int. Conf. Learning Representations*, 2018.
- [7] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. Int. Conf. Machine Learning*, 2021.
- [8] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems*, 2017.
- [9] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2018.
- [10] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2003.
- [11] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. Int. Conf. Learning Representations*, 2016.

A. Appendix

Supplementary material—extended plots, per-seed learning curves, and implementation notes—will be appended following the main experimental campaign.